Course Contents

Full Stack Java Course Contents

Audience: Learners preparing for entry-level Java / Spring Boot / full-stack roles
Stack: JDK 17 (21 mentioned) · Maven 3.9+ · MySQL 8+ · Spring Boot 3.x (Jakarta) · HTML/CSS/JS · React · GitHub · VS Code or IntelliJ Community · Postman
Duration (default plan): 20 weeks × 1 hour 15 minutes/day (Mon–Fri live) = **100 sessions / ~125 live hours** + homework
Schedule authority: `JAVA/Session_Calendar_100.md`
Practice DB: `training_db` | **IDE:** VS Code or IntelliJ Community | **VCS:** Git + GitHub
Legend: Must = job-ready path | *Optional* = deepen if hours allow / advanced batch

Syllabus source: KSR Datavizon Java Full Stack training (mandatory brochure topics preserved; sequence improved for classroom learning — same reasoning as Python FS).

> Live class length is **1h 15m**. One session = one focus. Required SQL labs live in repo `MYSQL/` (versioned external dependency). HTML/CSS/JS/React live in `FRONTEND/` — do not copy them here.

Program map (improved sequence)

Phase	Weeks	Focus	Outcome
0–1	1–2	Computer + Core Java foundations	Think like a programmer; compile and run small programs
2	3–5	Flow, strings, arrays, methods	Build reusable CLI tools
3	6–9	OOP (3 parts), collections, exceptions, files, Git	Intermediate Java + GitHub habit
4	10–12	MySQL (**required labs**) + JDBC	CRUD apps talking to MySQL (SQL first, then Java)
5	12–13	HTML, CSS, JavaScript + Spring Boot start	Front pages + first REST app
6	14–16	JPA, REST DTOs, security, labs `06`–`07`, capstone shell	Auth CRUD API + deploy checks
7	17–18	Servlet interview map + React	SPA consuming the Java API
8	19–20	Testing, capstone, placement (**AI optional**)	Portfolio + interview readiness

1. Computer & Programming Foundations (Must)

- What is a computer, software, OS, files/folders
- What is programming? Algorithms vs code
- How websites and apps work (client–server, browser, server)
- Roles: Java developer, Spring Boot developer, full-stack developer (India market)
- Tools: VS Code or IntelliJ Community, terminal/command prompt, browser DevTools overview

Practice: create folders, run terminal commands, open the IDE
Assignment: one-page “How a website works” flowchart

---

2. Core Java — Setup & Basics (Must)

- Introduction to Java & why India hiring uses it
- Install **JDK 17 LTS** (mention 21 LTS) + VS Code or IntelliJ Community
- `javac` / `java`; `public class` + `main`
- Keywords, identifiers, variables, primitive types
- Operators
- Input and output (`Scanner`, printing)
- Flow control: `if` / `else`, `switch`, `for`, `while`, `break` / `continue`

Practice: even/odd, grade calculator, multiplication table, number guessing
Assignments: Simple Calculator, ATM menu (CLI)

---

3. Core Java — Strings, Arrays, Methods (Must)

- String (immutability, methods, `equals` vs `==`)
- Arrays (1D, 2D, enhanced `for`)
- Static methods, return, parameters, overloading
- Data structures overview (array vs later `List`/`Map` — preview labelled Later)

Practice: student marks analysis, registration field cleaner
Mini project: Student Marks Manager (CLI, arrays)

---

4. OOP in Java (Must — heavier than Python OOP; three parts)

- Part 1: class, object, constructor, `this`, encapsulation
- Part 2: inheritance, `super()`, overriding
- Part 3: polymorphism, abstract classes, interfaces, packages
- Access modifiers; IS-A vs HAS-A (composition lite)

Mini project: BankAccount / campus wallet → push to GitHub
Course order note: Exceptions come **after** OOP. OOP labs validate with `print` + `return` (no `throw` / `try` / `catch` in the core path).

---

5. Collections, Exceptions, Files (Must)

- Collections: `ArrayList`, `HashSet`, `HashMap` (interfaces `List`/`Set`/`Map`)
- Exception handling: `try` / `catch` / `finally`, try-with-resources, custom exceptions

- File I/O (`java.nio` / streams) for CSV/text

Mini project: Expense tracker or inventory (collections + files)

---

6. Git & GitHub (Must)

Shared notes: `GIT/README.md` (same files as Python FS).

- `git init`, status, add, commit, log, branch, merge
- GitHub: repo, clone, push, pull, README, `.gitignore` (`target/` **and** `venv/`)
- Collaboration basics (PR overview)
- Five sessions in `GIT/`: first push, daily loop, branches/PRs, desk incidents, IDE (Sessions 3–5 extra lab if live time is only S44–S45)

Practice: push every mini project (`target/`, `.env`, `*.class` never committed)

---

7. Database (MySQL) (Must)

- Full-stack bridge + **required MySQL labs table**: `JAVA/06_Database_MySQL/01_MySQL_for_Full_Stack_Developers.md` (points into `MYSQL/` — mandatory for placement)
- DDL, DML, DQL, TCL, JOINS, normalization overview
- **Do not duplicate** `MYSQL/` into this folder

Practice: employee / student schema
Assignment: Library or E-commerce schema + CRUD queries
Detail: `MYSQL/Course_Contents.md` / `MYSQL/` modules

---

8. JDBC (Must)

- Connection, `PreparedStatement` (no string-concat SQL)
- SELECT / INSERT / UPDATE / DELETE
- try-with-resources; close resources
- Mini Library or student CRUD from Java

Next: Spring Data JPA comes **after** JDBC — learners must see SQL first.

---

9. Front-End Fundamentals (Must)

- **HTML:** structure, forms, tables, semantic tags — `FRONTEND/01_HTML.md`
- **CSS:** selectors, box model, Flexbox, responsive design — `FRONTEND/02_CSS_and_Responsive.md`
- **JavaScript:** variables, functions, DOM, events, forms, fetch basics — `FRONTEND/03_JavaScript_DOM_Forms.md`

Mini project: Responsive landing page + contact form validation
Taught before Spring so REST JSON and any Thymeleaf overview make sense.

---

10. Spring Boot (Must)

Home for Maven, Spring Web (REST-first), JPA, DTOs, security, and two protected lab days. JPA/REST are **not** a second copy under `10_JPA_and_REST/` (pointer only).

- Maven + Spring Boot intro — `09_Spring_Boot/01_…`
- Spring Web REST (controllers, JSON); Thymeleaf **short overview** — `02_…`
- Spring Data JPA + validation — `03_…`
- REST APIs, Postman, DTO mapping — `04_…`
- Spring Security lite (JWT lite, junior-realistic) — `05_…`
- **Protected lab:** CRUD + auth consolidation — `06_…`
- **Protected lab:** deploy + capstone debug — `07_…`
- **One session:** Servlet vs Spring interview map — `08_…` (not the backbone)

Capstone seed: CampusCart (Spring Boot 3 + MySQL + REST)

> Spring Boot package = **seven files** (`01` – `07`) plus one short interview-map session (`08`). Do not invent a Servlets/JSP track.

---

11. React (Must)

- Same shared notes as Python FS: `FRONTEND/REACT/`
- Connect the SPA to the **Java** Product API (CORS on Spring)

---

12. Testing, Debugging, Clean Code (Must)

- Debugging in VS Code / IntelliJ
- JUnit 5 basics; Mockito lite
- Clean-code habits for juniors (explicit getters or records; Lombok is Stretch only)

---

13. AI Integration (*Optional stretch / brochure*)

- **Only after** deployed Spring Boot MVP + SQL confidence + GitHub portfolio
- One AI API call from Spring (never blocks placement)

---

14. Miscellaneous — Soft Skills & Placement (Must)

- Agile (Scrum) overview
- Daily assignments & biweekly assessments
- Resume and LinkedIn optimization
- GitHub portfolio
- Mock interviews, aptitude/coding prep
- Live / capstone project
- Job assistance process (India entry-level Java / Spring Boot / full-stack roles)

Suggested weekly load

Mode	Weeks	Live hours/day	Sessions	Approx. live total
Default	**20**	**1h 15m**	**100**	**~125h** (+ homework)

Mode	Weeks	Live hours/day	Sessions	Approx. live total
Comfortable	22–24	1h 15m	110–120	Extra Spring/React buffer
Compressed	16	1h 15m + Sat lab	80+	Only with weekend labs

Adjust weeks only via the Session Calendar; keep module order fixed. **Do not plan as 4-hour daily classes.**